

1. Enunciado

En este proyecto se pretende construir un sistema de gestión de trabajo colaborativo a través de tableros de tareas. Como referencia para identificar las características de este tipo de aplicaciones se pueden explorar aplicaciones como Trello (<https://www.trello.com/>).

La aplicación deberá permitir:

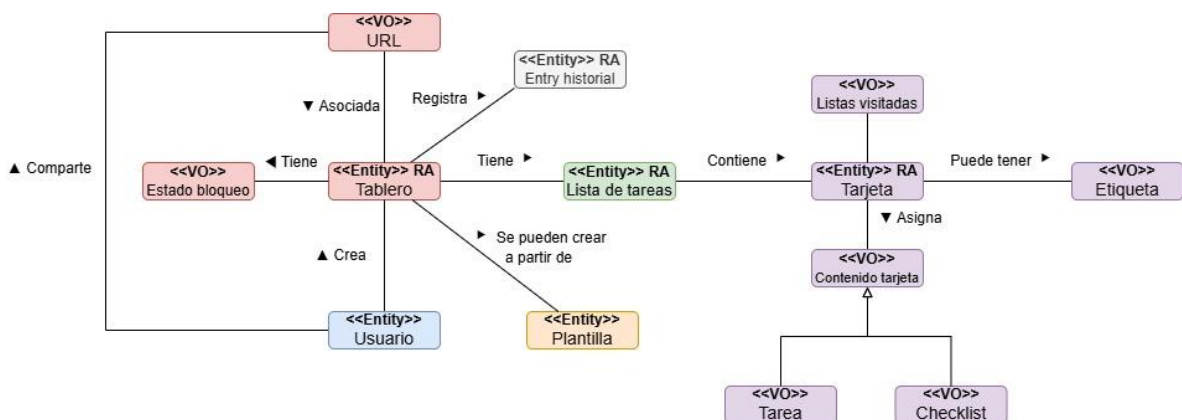
- Crear y modificar *tableros* donde hay *listas de tareas*. Una lista de tareas tiene *tarjetas* de manera que las tarjetas sirven para asignar tareas o para anotar información relevante.
- Las tarjetas se pueden marcar como completadas, en cuyo caso pueden pasar a una lista especial del tablero de tarjetas completadas.
- Las tarjetas pueden tener etiquetas para clasificarlas. Una etiqueta además tiene un color.
- Hay dos tipos de tarjetas: tarjetas que tienen tareas y tarjetas que tienen checklists.
- Los tableros mantienen una historia de todas las acciones de los usuarios. Por ejemplo, si se mueve una tarjeta entre listas de tareas se debe registrar una traza de que este movimiento se ha realizado.
- Es posible bloquear temporalmente un tablero para que no se puedan añadir nuevas tarjetas, solo mover tarjetas entre sus listas (por ejemplo, un tablero con TODO, DOING, DONE en el que durante una semana no se pueden añadir nuevas tarjetas).
- Para simplificar la gestión de los usuarios se utilizará una estrategia simple: cualquier persona con un correo electrónico podrá crear un tablero y obtendrá una URL única (y privada a no ser que la comparta) para su tablero.
- Un tablero puede compartirse con otros usuarios de la aplicación, para ello se comparte la URL.
- Implementar reglas a nivel de tablero o de lista de tareas:
 - Una lista no puede tener más de N ítems (configurable)
 - Una lista define que una tarjeta tiene que haber pasado por otras listas antes de llegar a ella.
- Filtrado de tarjetas por etiquetas.
- Creación de plantillas para tableros.
 - Las plantillas se definen como un fichero YAML.

- Autenticación basada en código por correo.

2. Lenguaje ubicuo

Término	Descripción
Tablero	Espacio de trabajo en el que colocar listas de tareas
Tarea	Una acción del proyecto que puede ser asignada a una tarjeta
Lista de tareas	Lista que contiene tarjetas
Tarjeta	Componente de una lista de tareas que almacena una tarea o anota información relevante
Etiqueta	Cada etiqueta es de un color y sirve para clasificar tarjetas
Checklist	Conjunto de tareas que, una vez completo, completan la tarjeta
TODO	Representa el estado de una tarea pendiente
DOING	Representa el estado de una tarea en proceso
DONE	Representa el estado de una tarea completada
Plantilla	Fichero que permite copiar la estructura y reglas de un tablero para construir uno nuevo
Historial	Registro de actividades en el tablero
Entrada historial	Cada uno de los eventos registrados en el historial

3. Modelo conceptual



4. Historias de usuario

4.1. Épica A → Gestión de tableros

HA1 – Crear tablero

Como usuario con email

Quiero crear un tablero nuevo

Para disponer de un espacio con listas donde gestionar tareas

Criterios de aceptación

- * Al crear un tablero, se genera una URL única para el tablero
- * Se puede inicializar el tablero vacío o desde una plantilla YAML
- * Devuelve ID del tablero y URL

HA2 – Bloquear tablero

Como usuario con acceso al tablero

Quiero configurar un bloqueo temporal del tablero

Para no poder añadir nuevas tarjetas y solo moverlas entre las listas del tablero

Criterios de aceptación

- * Puedo activar/desactivar bloqueo con rango de fechas (desde - hasta)
- * Los cambios quedan registrados creando una Entry Historial

HA3 – Desbloquear tablero

Como usuario con acceso al tablero

Quiero desactivar el bloqueo del tablero

Para trabajar con la configuración de bloqueo libremente

Criterios de aceptación

- * El tablero deja de estar bloqueado instantáneamente, y no espera hasta la fecha final del bloqueo establecido
- * Se permite añadir tarjetas libremente mientras que no se incumplan el resto de invariantes independientes al bloqueo como el límite N
- * El desbloqueo queda registrado creando una Entry Historial

HA4 – Compartir tablero

Como usuario con acceso al tablero

Quiero compartir la URL del tablero con otros usuarios

Para que puedan acceder y trabajar sobre el mismo tablero

Criterios de aceptación

- * Quien tenga la URL puede acceder y editar el tablero
- * La URL es única para cada tablero

HA5 – Modificar nombre del tablero

Como usuario con acceso al tablero

Quiero modificar el nombre del tablero

Para tener una mejor gestión de los diferentes espacios de trabajo

Criterios de aceptación

- * El tablero adquiere el nuevo nombre introducido
- * La URL no cambia ni los usuarios pierden acceso a pesar de haber cambiado de nombre
- * La modificación queda registrada con una Entry Historial
- * No se permite un nombre vacío o con caracteres inválidos y devuelve error de validación en ese caso

HA6 – Eliminar tablero

Como usuario con acceso al tablero

Quiero eliminar completamente un tablero

Para deshacerme de espacios de trabajo innecesarios

Criterios de aceptación

- * El tablero desaparece de entre los tableros disponibles
- * La URL asociada ya no da acceso al tablero
- * El historial del tablero desaparece y ya no es accesible

HA7 – Configurar prerequisites número de tarjetas por lista de tablero

Como usuario con acceso al tablero

Quiero definir que las listas no puedan contener más de N tarjetas (configurable)

Para evitar sobrecargar el flujo de tareas y trabajo

Criterios de aceptación

- * Si N se alcanza, añadir una tarjeta a la lista falla
- * Si una tarjeta se va fuera de la lista y queda hueco, se puede añadir otra
- * El límite puede modificarse y se registra la modificación creando una Entry Historial
- * La lista especial no se configura con límite N
- * Si alguna lista tenía ya definido un límite, este se sobrescribe
- * No se puede configurar el límite si alguna lista (excepto la especial) tiene más tarjetas que el límite N

4.2. Épica B → Listas y reglas por lista

HB1 – Crear/editar/eliminar listas

Como usuario con acceso al tablero

Quiero crear, renombrar y eliminar listas dentro del tablero

Para organizar el flujo de tareas y trabajo

Criterios de aceptación

- * Las listas tienen un nombre único en el tablero
- * La creación, modificación y eliminación de listas crean una nueva Entry Historial
- * Borrar una lista borrará todo el contenido dentro de esa lista

HB2 – Configurar límite de N ítems en una lista concreta

Como usuario con acceso al tablero

Quiero poner un límite de tarjetas N (configurable) en una lista

Para evitar sobrecargar el flujo de tareas y trabajo

Criterios de aceptación

- * Si N se alcanza, añadir una tarjeta a la lista falla
- * Si una tarjeta se va fuera de la lista y queda hueco, se puede añadir otra
- * El límite puede modificarse y se registra la modificación con una Entry Historial
- * Intentar configurar límite N en una lista especial falla
- * No se puede configurar el límite si la lista (excepto la especial) tiene más tarjetas que el límite N

HB3 – Configurar prerequisites de lista

Como usuario con acceso al tablero

Quiero definir que para entrar en la lista C una tarjeta debe haber pasado por listas A y B antes

Para imponer orden en el flujo de trabajo

Criterios de aceptación

- * Intentar mover una tarjeta a lista C sin haber pasado por las listas requeridas falla
- * La lista de visitados de la tarjeta mantiene las listas por las que ha pasado la tarjeta
- * Esta acción registra una Entry Historial
- * Intentar configurar prerequisites falla si alguna de las tarjetas ya presente en la lista no cumple los requisitos

HB4 – Crear lista especial

Como usuario con acceso al tablero

Quiero definir que una lista es especial de tarjetas completadas

Para reflejar un flujo ordenado y organizado de trabajos completados

Criterios de aceptación

- * Si ya hay una lista especial creada anteriormente, la creación fallará
- * Las tarjetas marcadas como completadas se mueven a esta lista
- * El criterio a nivel de tablero de límite N no afecta a esta lista
- * Se crea una nueva Entry Historial que indique la configuración de la lista especial

4.3. Épica C → Tarjetas

HC1 – Crear tarjeta (tarea/checklist)

Como usuario con acceso al tablero

Quiero añadir una tarjeta a una lista con contenido (tarea o checklist)

Para capturar un trabajo o información relevante

Criterios de aceptación

- * Se puede crear la tarjeta como una tarea o como una checklist que contenga una lista de ítems
- * Si el tablero está bloqueado para añadir, la operación falla
- * La tarjeta obtiene un ID único y la creación provoca una nueva Entry Historial
- * La tarjeta se crea en la última posición de la lista
- * Si es una checklist, tiene al menos un ítem

HC2 – Mover tarjeta entre listas

Como usuario con acceso al tablero

Quiero mover tarjetas entre listas

Para reflejar el progreso del trabajo

Criterios de aceptación

- * Los movimientos válidos se permiten, a no ser que incumpla prerequisites de la lista como el límite N
- * Cada movimiento genera una nueva Entry Historial
- * Si el tablero está bloqueado, mover tarjetas está permitido

HC3 – Marcar tarjeta como completada

Como usuario con acceso al tablero

Quiero marcar una tarjeta como completada

Para que pase a una lista concreta especificada

Criterios de aceptación

- * La acción mueve la tarjeta a la lista especial
- * Si en una checklist todos los ítems han sido marcados como completados, la tarjeta se completa automáticamente
- * Si la tarjeta no ha pasado por las listas necesarias antes de pasar a la lista especial, la operación falla
- * Si no existe lista especial de completadas, la operación falla y se lanza un error de dominio indicando que no hay lista especial especificada

HC4 – Añadir/eliminar/modificar etiquetas en tarjeta

Como usuario con acceso al tablero

Quiero añadir etiquetas (color + nombre) a las tarjetas

Para poder filtrarlas y clasificarlas

Criterios de aceptación

- * La etiqueta se añade individualmente a cada tarjeta
- * Se pueden añadir múltiples etiquetas a una tarjeta
- * Filtrado por etiqueta devuelve tarjetas que contengan esa etiqueta
- * La creación, eliminación o modificación de las etiquetas se ve reflejada en una Entry Historial

HC5 – Editar contenido de la tarjeta

Como usuario con acceso al tablero

Quiero editar la descripción de la tarea o modificar la checklist de ítems

Para mantener la información actualizada

Criterios de aceptación

- * Se reemplaza el contenido de la tarjeta
- * Cada edición genera una nueva Entry Historial

HC6 – Renombrar tarjeta

Como usuario con acceso al tablero

Quiero editar el título de la tarjeta

Para gestionar libremente los nombres de las tarjetas

Criterios de aceptación

- * Se reemplaza el nombre de la tarjeta
- * Cada renombrado genera una nueva Entry Historial

4.4. Épica D → Plantillas

HD1 – Definir plantilla en YAML

Como usuario con acceso al tablero

Quiero crear una plantilla (fichero YAML) que describa listas y tarjetas iniciales y sus reglas

Para reutilizar configuraciones de tablero

Criterios de aceptación

- * La plantilla incluye un nombre, las mismas listas, tarjetas y reglas por lista que en el tablero que se ha creado la plantilla
- * El YAML se valida, error si no cumple el formato

HD2 – Crear tablero desde plantilla

Como usuario

Quiero crear un tablero a partir de una plantilla YAML

Para iniciar rápidamente la estructura con reglas

Criterios de aceptación

- * El tablero se crea con listas y reglas tal y como define la plantilla
- * Con una nueva Entry Historial se registra que el tablero se ha creado usando la plantilla

4.5. Épica E → Autenticación y seguridad

HE1 – Login por código email

Como usuario con correo

Quiero iniciar sesión introduciendo mi email y un código que me llega por correo

Para no gestionar contraseñas

Criterios de aceptación

- * La petición de login genera un código temporal enviado por email
- * El código expira pasado cierto tiempo

4.6. Épica F → Historial

HF1 – Registrar eventos en el historial del tablero

Como usuario con acceso al tablero

Quiero registrar en un historial las acciones sobre el tablero

Para monitorizar cambios y ver quién ha hecho cada acción

Criterios de aceptación

- * Cada acción generará una Entry Historial de un tipo concreto.
- * Cada registro lleva un timestamp, TableroId y los detalles del tipo de registro. Formato del registro ejemplo: EntryId, TableroId, tipo, usuario (email), timestamp, detalles (e.g. TarjetaMovida tarjetaId fromListId toListId)

HF2 – Consultar historial de un tablero

Como usuario con acceso al tablero

Quiero consultar el historial de acciones del tablero

Para saber qué cambios se han realizado, cuándo y quién los ha hecho

Criterios de aceptación

- * Se consulta correctamente el historial asociado a un TableroId
- * El resultado es paginado, conteniendo un máximo de 20 entradas por página
- * En cada entrada se observa EntryId, tipo, usuario (email), timestamp y detalles (e.g. TarjetaMovida tarjetaId fromListId toListId)

4.7. Épica G → Búsqueda y filtrado

HG1 – Filtrar tarjetas por etiqueta

Como usuario con acceso al tablero

Quiero ver solamente tarjetas que contienen X etiquetas

Para centrarme en un subconjunto de trabajo

Criterios de aceptación

- * Se puede filtrar por una o varias etiquetas a la vez (AND/OR configurable) especificando únicamente el nombre de la etiqueta
- * No aparecen tarjetas que no cumplan con los filtros de la búsqueda
- * Si no se configura AND/OR, por defecto se filtrarán con modo OR
- * Si no se indican etiquetas se devuelve un error de validación

4.8. Épica H → Reglas de negocio y validaciones (historias técnicas)

HH1 – Validar límite N al añadir/mover tarjetas

Como sistema

Quiero impedir acciones que violen el límite N de una lista

Para mantener el invariante de capacidad

Criterios de aceptación

* Las operaciones de añadir tarjeta y mover tarjeta comprueban el límite antes de aplicarse

* Si se incumple la regla, se lanza una excepción de dominio

HH2 – Validar prerequisites de listas al mover tarjeta

Como sistema

Quiero comprobar las listas visitadas de la tarjeta

Para validar prerequisites y garantizar el flujo definido

Criterios de aceptación

* La operación de mover tarjeta comprueba las listas ya recorridas por esa misma tarjeta

* Si no se cumple, la operación se rechaza con un mensaje que indica qué listas faltan

5. Reglas de negocio

- R1 – Cuando un tablero está bloqueado, durante ese tiempo no se pueden añadir nuevas tarjetas, solo moverlas entre sus listas.
- R2 – Cuando una lista está configurada con límite de ítems N, no se pueden añadir ni mover nuevas tarjetas a esta lista.
- R3 – Si una lista define que una tarjeta tiene que haber pasado por otras listas antes de llegar a ella, solo aceptará tarjetas que cumplan dicho prerequisite.
- R4 – Si existe lista especial, marcar tarjeta como completada mueve la tarjeta a esa lista. Si no existe, la operación falla y se lanza un error de dominio indicando que no hay lista especial completada.
- R5 – Toda acción que modifique el estado de un tablero, lista o tarjeta debe generar exactamente una Entry Historial que contenga: EntryId, TableroId, tipo, usuario (email), timestamp y detalles según el tipo de Entry.
- R6 – Una Entry Historial es append-only y no se modifica, es immutable. Solo se elimina en cascada cuando se elimina el tablero (hard delete).
- R7 – El historial consultado para un tablero es el conjunto ordenado de manera descendente por timestamp de Entry Historial con el mismo TableroId. Este historial está paginado.
- R8 – Cuando una tarjeta se crea, debe de crearse dentro de una lista y se añadirá en la última posición de la lista.
- R9 – Todas las listas que se configuren como prerequisites de una lista tienen que formar parte del tablero.

- R10 – Todas las listas dentro de un mismo tablero tienen que tener un nombre único.

6. Entidades y objetos valor

Definimos entidades a aquellos componentes del dominio que tienen identidad propia y que pueden mutar a lo largo de su ciclo de vida. Por otro lado, el value object se define por sus atributos y no tiene identidad propia, comúnmente tratándose como objetos inmutables y sustituyéndolos por otros nuevos en caso de ser modificados. Con esta definición, categorizamos cada elemento de nuestro dominio:

- **Tablero:** Entidad, puesto que tiene identidad propia y puede ir cambiando a lo largo de su ciclo de vida. Su nombre puede cambiar sin perder identidad y la URL nunca cambiará aunque se modifique el nombre.
- **Lista de tareas:** Entidad, ya que también tiene identidad propia y cada lista puede ser modificada. Tiene reglas propias asociadas y necesitamos distinguir entre unas listas y otras.
- **Tarjeta:** Entidad, ya que puede haber dos tarjetas con el mismo contenido y etiquetas y aún así ser distintas, tiene identidad.
- **Usuario:** Entidad por definición, tienen identidad, se identifican por el email.
- **Plantilla:** Entidad, porque cada plantilla es diferente, ya que se crea mediante un fichero YAML que copia la estructura de cada tablero con sus listas y tarjetas.
- **URL:** Value object, no tiene identidad propia en el dominio, solo importa el valor del enlace en sí.
- **Estado bloqueo:** Value object, simplemente importan sus atributos de saber si está activo y la fecha (desde cuándo hasta cuándo).
- **Entry historial:** Entidad, cada entrada tiene una identidad única y diferente. Las entradas que coinciden en el ID del tablero formarán el historial completo de dicho tablero. Cada entrada es de forma append-only, el historial está paginado y es accesible desde la interfaz de usuario. Aunque sea inmutable se sigue considerando entidad.
- **Listas visitadas:** Value object, no tiene identidad, es solamente un conjunto de valores inmutable que se utiliza para validar prerequisites.
- **Etiqueta:** Value object, importa solamente por su nombre y su color, son los atributos que observamos, no necesita de identidad propia. En el caso de considerar un catálogo de etiquetas disponible y gestionable, sí se podría tratar como una entidad, pero no lo modelaremos así.

- **Contenido tarjeta:** Value object, es simplemente el contenido dentro de la tarjeta, puede ser igual en varias tarjetas. No necesita identidad.
- **Tarea:** Value object, es un tipo concreto de Contenido tarjeta, vive dentro de cada tarjeta.
- **Checklist:** Value object, igual que la tarea, es un subtipo de Contenido tarjeta. Vive dentro de cada tarjeta y carece de identidad.

7. Agregados

El concepto de agregado se define como un conjunto de objetos del dominio que se tratan como una unidad de consistencia y cuya coherencia se protege mediante reglas del dominio.

Nuestro DDD consta de 6 agregados que cobran sentido, mantienen la coherencia y preservan las reglas del dominio al ser agrupados:

- **Agregado Tablero – Tablero como raíz del agregado** → El primer agregado que consideramos es el de Tablero, el cual representa el espacio de trabajo y contiene las **políticas globales** de las listas y tarjetas y la configuración que afecta al comportamiento del sistema. Es importante recalcar que no contiene directamente las listas ni las tarjetas como parte de su agregado, sino que actúa únicamente como contexto que define las reglas que deben de cumplirse en otros agregados. Es responsable de:
 - Gestionar la URL única de acceso
 - Mantener el estado de bloqueo
 - Configurar reglas globales y por lista
 - Garantizar coherencia entre las políticas de los invariantes

Las operaciones que implican modificar simultáneamente listas o tarjetas (como mover una tarjeta o eliminar una lista con todas sus tarjetas) se coordinan desde los servicios de aplicación aceptando consistencia eventual entre agregados, condición aceptable para este sistema.

- **Agregado Lista de tareas – Lista como raíz del agregado** → Este agregado representa un espacio dentro del tablero que contiene tarjetas. Cada lista tiene identidad propia y está asociada a un identificador de tablero, pero mantiene su propio límite transaccional. Es responsable de:
 - Mantener el conjunto de las tarjetas contenidas y su orden

- Aplicar el límite N de tarjetas por lista si está configurado
- Garantizar que no se exceda la capacidad configurada
- Gestionar la eliminación en cascada de las tarjetas que contiene cuando se elimina una lista

Las operaciones que implican coordinación entre listas y tarjetas (como por ejemplo mover una tarjeta entre listas) se gestionan mediante los servicios de aplicación, aceptando de nuevo consistencia eventual entre agregados.

- **Agregado Tarjeta – Tarjeta como raíz del agregado** → Representa la unidad de información dentro de la una lista. Cada tarjeta tiene identidad propia y mantiene un ciclo de vida independiente al resto de agregados. Es responsable de:
 - Gestionar el contenido propio dentro de la tarjeta
 - Mantener las etiquetas asignadas a cada tarjeta
 - Registrar la lista actual a la que pertenece y las listas por las que ha pasado para validar prerequisites
 - Cambiar su estado (moverse entre listas, editar el contenido, eliminar la tarjeta o marcarla como completada, que será en esencia, moverla a una lista especial)

Este agregado Tarjeta no tiene la responsabilidad de validar las reglas globales del tablero. Dichas validaciones se realizan consultando el agregado del Tablero mediante servicios de aplicación. Al separar Tarjeta y considerarlo un agregado independiente se permite mayor concurrencia, se reduce el tamaño del agregado Tablero y favorece la escalabilidad.

- **Agregado Usuario – Usuario como raíz del agregado** → Su identidad y su ciclo de vida es propio e independiente al tablero. Un mismo usuario puede crear o acceder a varios tableros, lo que lo hace independiente. Se hace referencia a él desde otros agregados mediante el correo electrónico del usuario.
- **Agregado Plantilla – Plantilla como raíz del agregado** → La plantilla es una entidad reutilizable que es **independiente** del tablero. La plantilla se crea una vez y se puede usar para crear **muchos tableros**, con las mismas listas, reglas y tarjetas, pero diferentes en esencia y con ciclos de vida distintos. También sigue sus propias invariantes, como el cumplimiento del esquema correcto del YAML. La coherencia entre las reglas de un tablero y la plantilla serán **heredadas** del propio tablero, pero pasarán a formar parte de las características de la plantilla. Dicho de otra forma, se copia la estructura del tablero pero se desentiende de aquel tablero a partir del cual se ha creado la plantilla.

- **Agregado Entry Historial – Entrada Historial como raíz del agregado** → Este agregado representa una traza inmutable de una acción que ha ocurrido en el tablero. El principal objetivo es permitir la consulta del historial de actividades de cada tablero sin necesidad de cargar el agregado tablero con toda su información. El historial de un tablero se construye como la consulta de todas las Entrada Historial cuyo TableroId coincide con el tablero que se consulta. Se modela como agregado independiente además porque tiene un patrón de acceso distinto: es append-only para la escritura y consulta paginada para la lectura, mientras que el tablero necesita consistencia sobre las listas y las tarjetas.

8. Servicios de aplicación

Los servicios de dominio contienen lógica que no pertenece exclusivamente a una sola entidad o value object o que orquesta varias responsabilidades.

- **ServicioTablero** → Maneja las operaciones complejas sobre el agregado tablero. Aunque muchos de estos métodos se ejecutan en métodos dentro del propio agregado, este servicio de dominio actúa como fachada entre casos de uso (valida precondiciones de aplicación, inicia la transacción, invoca métodos del agregado, provoca que se creen las entries del historial...)

```
public interface ServicioTablero {
    // Se trata de una operación compleja, por lo que se usa un Command
    ResultadoCrearTableroDTO crearTablero(CrearTableroCmd cmd);
    void renombrarTablero(String tableroId, String nombreNuevo, String emailUsuario);
    void eliminarTablero(String tableroId);

    void bloquearTablero(String tableroId, LocalDateTime desde, LocalDateTime hasta, String motivo, String emailUsuario);
    void desbloquearTablero(String tableroId, String emailUsuario);
    // Configurar límite N a nivel de tablero
    void configurarLimiteTablero(String tableroId, Integer limite, String emailUsuario);
}
```

- **ServicioLista** → Igual que el ServicioTablero, es el orquestador responsable de las operaciones sobre listas que involucran a varios agregados.

```
public interface ServicioLista {
    ListaDTO crearLista(String tableroId, String nombre, String emailUsuario);
    void renombrarLista(String tableroId, String listaId, String nuevoNombre, String emailUsuario);
    void eliminarLista(String tableroId, String listaId, String emailUsuario);
    void definirListaEspecial(String tableroId, String listaId, String emailUsuario);
}
```

```

        // Configuramos el número de elementos que puede tener una determinada
        lista
        void configurarLimiteLista(String tableroId, String listaId, Integer
        limite, String emailUsuario);
        // Configuramos los prerequisites de una lista
        void configurarPrerrequisitosLista(String tableroId, String listaId,
        Set<String> prerrequisitos, String emailUsuario);
    }

```

- ServicioTarjeta → El tercero de los servicios core de la aplicación, también responsable de manejar las operaciones complejas sobre tarjetas.

```

public interface ServicioTarjeta {
    // La tarjeta se crea dentro de una lista
    TarjetaDTO crearTarjeta(String tableroId, String listaId, String nombre,
    ContenidoTarjetaCmd cmd);
    // Modificar el contenido de la tarjeta. Se puede cambiar de tarea a
    checklist o viceversa
    void editarContenidoTarjeta(String tableroId, String listaId, String
    tarjetaId, ContenidoTarjetaCmd cmd);
    void renombrarTarjeta(String tableroId, String listaId, String tarjetaId,
    String nuevoNombre, String emailUsuario);
    void eliminarTarjeta(String tableroId, String listaId, String tarjetaId,
    String emailUsuario);
    // Mover la tarjeta a una lista concreta
    void moverTarjeta(String tableroId, String tarjetaId, String
    listaOrigenId, String listaDestinoId, String emailUsuario);
    // Marcar una tarjeta como completada (en realidad es como moverla a la
    lista especial)
    void completarTarjeta(String tableroId, String listaId, String tarjetaId,
    String emailUsuario);
    // Marcar como completado item de la checklist y completar la tarjeta si
    procede
    void completarItemChecklist(String tableroId, String listaId, String
    tarjetaId, int indiceItem, String emailUsuario);
    // Desmarcar item de checklist
    void marcarItemChecklistComoPendiente(String tableroId, String listaId,
    String tarjetaId, int indiceItem, String emailUsuario);

    // ETIQUETAS
    // Añadir una etiqueta (nombre y color) a una tarjeta de un tablero
    void addEtiquetaATarjeta(String tableroId, String listaId, String
    tarjetaId, String nombre, String color, String emailUsuario);
    void eliminarEtiquetaDeTarjeta(String tableroId, String listaId, String
    tarjetaId, String nombre, String color, String emailUsuario);
    void modificarEtiquetaEnTarjeta(String tableroId, String listaId, String
    tarjetaId, String nombreOld, String colorOld, String nombreNuevo, String
    colorNuevo, String emailUsuario);
}

```

- ServicioPlantilla → Responsable de validar y parsear el fichero YAML, guardar plantillas y crear tableros a partir de plantillas.

```

public interface ServicioPlantilla {

    PlantillaDTO crearPlantilla(String yaml, String emailUsuario);
}

```

```
PlantillaDTO obtenerPlantilla(String plantillaId);
}
```

- ServicioHistorial → Actúa como servicio de aplicación de lectura o fachada, no como lógica pura del dominio.

```
public interface ServicioHistorial {
    PageDTO<EntryHistorialDTO> consultarPorTablero(String tableroId, int
pagina, int tamano);
}
```

- ServicioAutenticacion → Genera y valida los códigos de login y los envía por correo. Es un servicio de consulta.

```
public interface ServicioAutenticacion {
    void enviarCodigoLogin(UsuarioId email);
    String verificarCodigoLogin(UsuarioId email, String codigo);
}
```

- ServicioSesion → El servicio valida que en cada request se valide el token de la sesión. Se llama desde un filtro HTTP (o podría también llamarse desde cada controlador antes de cada caso de uso). Aunque se podría incluir dentro de ServicioAutenticacion, lo separamos para diferenciar mejor la semántica en el DDD.

```
public interface ServicioSesion {
    public UsuarioId validarYRenovarToken(String token);
}
```

- ServicioFiltradoTarjetas → Realiza la consulta de tarjetas filtrando por etiquetas asignadas a dichas tarjetas. También es un servicio de consulta.

```
public interface ServicioFiltradoTarjetas {

    PageDTO<TarjetaDTO> filtrarPorEtiquetas(String tableroId, List<String>
nombresEtiquetas, ModoFiltradoEtiquetas modo,
int pagina, int tamano);
}
```

9. Servicios de dominio

Los servicios de dominio contienen lógica pura de negocio. Pertenecen en general al modelo de dominio y opera sobre diferentes entidades, value objects y agregados. Deben su existencia a que la lógica no encaja de manera natural dentro de una sola entidad. Para este proyecto, tenemos dos servicios de dominio: PoliticaLista y PoliticaTarjetas que validan las reglas de negocio que actúan sobre los elementos del dominio:

```
public class PoliticaListas {

    // Validar que todas las listas que se quieran configurar como
prerrequisito existan en el tablero (R10)
    public void validarPrerrequisitosConfigurados(Set<ListaId> listasTablero,
Set<ListaId> prerrequisitos);
}
```



```

    // Validar que todas las listas de un tablero tienen nombre único (R11)
    public void validarNombreUnicoEnTablero(TableroId tablero, String
nombrelista);
}

```

```

public class PoliticaTarjetas {

    // Hace uso de los métodos validarBloqueo, validarLimite y
validarCrearEnListaConPrerrequisitos para
    // validar todas las reglas de negocio a la hora de crear la tarjeta (R1,
R2, HH1)
    public void validarCreacion(Tablero tablero, Lista listaDestino);

    // Hace uso de los métodos validarLimite y validarPrerrequisitos para
validar todas las reglas de negocio
    // a la hora de mover una tarjeta de una lista a otra. Si el tablero está
bloqueado se pueden mover las tarjetas (R2, HH1)
    public void validarMovimientoEntreListas(Tarjeta tarjeta, Lista
listaDestino);

    // En esencia, mover una tarjeta a la lista especial
    public void validarCompletar(Lista listaEspecial, Tarjeta tarjeta);

    // Comprobar que las tarjetas de una lista cumplen con los prerrequisitos
antes de ser configurados (HB3)
    public void validarConfiguracionPrerrequisitos(List<Tarjeta>
tarjetasLista, Set<ListaId> prerrequisitos);
}

```

No forma parte del DDD pero es necesario documentar:

10. Puertos / Repositorios (Interfaces de salida)

Definiremos estas interfaces en la capa de dominio/aplicación (puertos):

```

package InterfacesSalida;

/* ----- TABLEROS ----- */

interface RepositorioTableros {
    void guardar(Tablero tablero);

    Optional<Tablero> buscarPorId(TableroId tableroId);

    Optional<Tablero> buscarPorURL(String tokenURL);

    // Necesario para HA6 (hard delete)
    void eliminarPorId(TableroId tableroId);

    // Útil para tableros disponibles (HA6)
    List<TableroId> listarIdsPorUsuario(UsuarioId usuarioId);
}

```

```

}

/* ----- PLANTILLAS ----- */

interface RepositorioPlantilla {
    void guardar(Plantilla plantilla);

    Optional<Plantilla> buscarPorId(PlantillaId plantillaId);
}

/* ----- LISTAS ----- */

interface RepositorioListas {
    void guardar(Lista lista);

    Optional<Lista> buscarPorId(ListaId listaId);

    // Útil para configurar límite a nivel de tablero, configurar prerequisites
y eliminar tablero
    Set<Lista> buscarPorIds(Set<ListaId> ids);

    // Necesario para HA6 (hard delete cascada) y para operaciones por tablero
    Set<Lista> buscarPorTableroId(TableroId tableroId);

    // Necesario para borrar lista (HB1) y cascada al borrar tablero
    void eliminarPorId(ListaId listaId);

    // Útil para borrar todo por tablero sin cargarlo (hard delete)
    void eliminarPorTableroId(TableroId tableroId);
}

/* ----- TARJETAS ----- */

enum ModoFiltradoEtiquetas { AND, OR }

interface RepositorioTarjetas {
    void guardar(Tarjeta tarjeta);

    Optional<Tarjeta> buscarPorId(TarjetaId tarjetaId);

    // Necesario para eliminar todas las tarjetas dentro de una lista (HB1)
    List<Tarjeta> buscarPorListaId(ListaId listaId);

    // Necesario para hard delete cascada al eliminar tablero
    List<Tarjeta> buscarPorTableroId(TableroId tableroId);

    // Borrado directo
    void eliminarPorId(TarjetaId tarjetaId);

    // Borrados masivos
    void eliminarPorListaId(ListaId listaId);

    void eliminarPorTableroId(TableroId tableroId);

    // HG1 - Filtrado por etiquetas (consulta)

```

```

        Page<Tarjeta> filtrarPorEtiquetas(
            TableroId tableroId,
            List<String> nombresEtiquetas,
            ModoFiltradoEtiquetas modo,
            PageRequest pageRequest
        );
    }

    /* ----- USUARIOS ----- */

    interface RepositorioUsuarios {
        void guardar(Usuario usuario);

        Optional<Usuario> buscarPorEmail(UsuarioId usuarioId);
    }

    /* ----- HISTORIAL (EntryHistorial) -----
    - */

    interface RepositorioEntryHistorial {
        void guardar(EntryHistorial entry);

        Page<EntryHistorial> consultarPorTablero(TableroId tableroId, PageRequest
pageRequest);

        // Necesario para HA6 (hard delete) -> "El historial del tablero desaparece"
        void eliminarPorTableroId(TableroId tableroId);
    }

    /* ----- EMAIL (autenticación) ----- */

    interface PuertoEnvioEmail {
        void enviarEmail(UsuarioId destinatario, String asunto, String cuerpo);
    }

    /* ----- YAML (plantillas) ----- */

    interface PuertoParserYAML {
        EspecificacionTableroPlantilla parse(String yaml);
    }

    /* ----- CÓDIGOS DE LOGIN ----- */

    interface RepositorioCodigosLogin {
        void guardarCodigo(UsuarioId usuarioId, String codigo, Instant expiraEn);

        Optional<String> buscarCodigoVigente(UsuarioId usuarioId);

        void invalidarCodigo(UsuarioId usuarioId);
    }

    /* ----- SESIONES DE APLICACIÓN ----- */
    public interface RepositorioSesiones {

        void guardarToken(String token, UsuarioId usuarioId, Instant expiraEn);
    }

```

```

Optional<UsuarioId> buscarUsuarioPorTokenVigente(String token);

void extenderExpiracion(String token, Instant nuevaExpiracion);

void invalidarToken(String token);
}

```

Esto nos permitirá implementar los adapters concretos en Spring Boot.

11. Eventos del dominio

Aquí consideramos todos los eventos de dominio importantes y relativos al sistema que vamos a construir. Muchos de ellos generan registros de tipo Entry Historial.

```

    TableroCreado(TableroId tableroId, UsuarioId usuarioId, LocalDateTime
timestamp, String nombreTablero);
    TableroCreadoDesdePlantilla(TableroId tableroId, UsuarioId usuarioId,
LocalDateTime timestamp, String nombreTablero, String nombrePlantilla);
    TableroEditado(TableroId tableroId, UsuarioId usuarioId, LocalDateTime
timestamp, String nombreAnterior, String nombreNuevo);
    TableroEliminado(TableroId tableroId, UsuarioId usuarioId, LocalDateTime
timestamp, String nombreTablero);
    TableroBloqueado(TableroId tableroId, UsuarioId usuarioId, LocalDateTime
timestamp, String motivo);
    TableroDesbloqueado(TableroId tableroId, UsuarioId usuarioId,
LocalDateTime timestamp);

    ListaCreada(ListaId listaId, TableroId tableroId, UsuarioId usuarioId,
LocalDateTime timestamp, String nombreLista);
    ListaEditada(ListaId listaId, TableroId tableroId, UsuarioId usuarioId,
LocalDateTime timestamp, String nombreAnterior, String nombreNuevo);
    ListaEliminada(ListaId listaId, TableroId tableroId, UsuarioId usuarioId,
LocalDateTime timestamp, String nombreLista);
    LimiteListaConfigurado(ListaId listaId, TableroId tableroId, UsuarioId
usuarioId, LocalDateTime timestamp, Integer limiteAnterior, Integer limiteNuevo,
String nombreLista);
    ListaEspecialDefinida(ListaId listaId, TableroId tableroId, UsuarioId
usuarioId, LocalDateTime timestamp, String nombreLista);
    // "Una lista define que una tarjeta tiene que haber pasado por otras
listas antes"
    PrerrequisitosListaConfigurados(ListaId listaId, TableroId tableroId,
UsuarioId usuarioId, LocalDateTime timestamp, Set<PrerrequisitoInfo>
prerrequisitos, String nombreLista);

    TarjetaCreada(TarjetaId tarjetaId, ListaId listaId, TableroId tableroId,
UsuarioId usuarioId, LocalDateTime timestamp, String nombreTarjeta, String
nombreLista);
    TarjetaEditada(TarjetaId tarjetaId, String nombreTarjeta, ListaId
listaId, TableroId tableroId, UsuarioId usuarioId, LocalDateTime timestamp,
ContenidoTarjeta contenidoAnterior, ContenidoTarjeta contenidoNuevo);

```

```
    TarjetaRenombrada(TarjetaId tarjetaId, ListaId listaId, TableroId
tableroId, UsuarioId usuarioId, LocalDateTime timestamp, String nombreAnterior,
String nombreNuevo);
    TarjetaEliminada(TarjetaId tarjetaId, ListaId listaId, TableroId
tableroId, UsuarioId usuarioId, LocalDateTime timestamp, String titulo);
    TarjetaMovida(TarjetaId tarjetaId, ListaId listaOrigenId, String
listaOrigen, ListaId listaDestinoId, String listaDestino, TableroId tableroId,
UsuarioId usuarioId, LocalDateTime timestamp, String nombreTarjeta);
    // Realmente es mover una tarjeta a la lista especial
    TarjetaCompletada(TarjetaId tarjetaId, ListaId listaId, String
nombrelista, TableroId tableroId, UsuarioId usuarioId, LocalDateTime timestamp,
String nombreTarjeta);
    EtiquetaAnadidaATarjeta(TarjetaId tarjetaId, ListaId listaId, TableroId
tableroId, UsuarioId usuarioId, LocalDateTime timestamp, Etiqueta etiqueta,
String nombreTarjeta);
    EtiquetaEliminadaDeTarjeta(TarjetaId tarjetaId, ListaId listaId,
TableroId tableroId, UsuarioId usuarioId, LocalDateTime timestamp, Etiqueta
etiqueta, String nombreTarjeta);
    // Como son VO, realmente consta de una eliminación y una adición
    EtiquetaModificadaEnTarjeta(TarjetaId tarjetaId, ListaId listaId,
TableroId tableroId, UsuarioId usuarioId, LocalDateTime timestamp, Etiqueta
etiquetaAnterior, Etiqueta etiquetaNueva, String nombreTarjeta);
```